

AI-Based Smart Notes Summarizer for Efficient Academic Learning

1st Lakshay
Computer Science
K. R. Manglam University
Gurgaon, Haryana
2301010306@krmu.edu.in

2nd Yash Chauhan
Computer Science
K. R. Manglam University
Gurgaon, Haryana
2301010308@krmu.edu.in

3rd Sagar
Computer Science
K. R. Manglam University
Gurgaon, Haryana
2301010312@krmu.edu.in

4th Rishit Garg
Computer Science
K. R. Manglam University
Gurgaon, Haryana
2301010334@krmu.edu.in

5th Yash Singh
Computer Science
K. R. Manglam University
Gurgaon, Haryana
2301010339@krmu.edu.in

Abstract—We are all aware of the importance of academic notes, textbooks, and PDF study materials in a student’s learning journey. One of the most significant challenges faced by students today is studying and revising large volumes of content in limited time while also finding good-quality notes. Many students depend on informal sources such as friends or online groups, which are often unorganized and unreliable. Due to increasing syllabus load and tight exam schedules, students struggle to read lengthy documents and identify important points efficiently.

This paper proposes an AI-Based Smart Notes Sharing and Summarization Platform for Academic Learning. The system allows students to upload, share, organize, and rate study notes while also generating concise summaries and key bullet points using generative AI. The summarized content is stored securely in a database, enabling quick access for revision. The proposed system improves learning efficiency, supports collaborative learning, and reduces time spent on manual summarization. Future enhancements include OCR-based handwritten note support, multilingual summarization, and AI-based question-answer generation.

Index Terms—Artificial Intelligence, Text Summarization, NLP, Web Application, Academic Learning

I. INTRODUCTION

In today’s academic environment, students are expected to learn and revise large amounts of information within limited time. Most study materials are available in digital formats such as PDFs, e-books, and research papers. While these resources provide detailed knowledge, they also create challenges due to their length and complexity.

Students often struggle to identify key points and spend excessive time reading unnecessary details. Manual note-making is time-consuming and inefficient. Existing digital tools lack automated summarization and structured revision support.

To overcome these issues, an AI-based Smart Notes Summarizer is proposed. It uses Natural Language Processing (NLP) to generate concise summaries and key points, helping students learn faster and revise efficiently.

II. MOTIVATION

The rapid growth of digital education and increasing academic pressure have made learning more demanding. Students must study large volumes of content within limited time, leading to stress and reduced productivity.

Manual summarization requires significant effort and repetition. Students often need only key concepts, but they must read entire documents to extract them. AI-based tools provide an opportunity to automate this process.

The motivation behind this project is to develop a system that reduces manual effort, improves learning efficiency, and provides structured academic summaries.

III. LITERATURE REVIEW

A. AI-Based Summarization in Education

AI summarization tools improve reading comprehension and reduce manual effort in note preparation. Studies show enhanced learning outcomes and increased student motivation.

B. Automatic Text Summarization

Automatic summarization converts long text into concise summaries while preserving meaning. It is widely used to handle large information efficiently.

C. Generative AI-Based Systems

Recent advancements in generative AI enable high-quality summaries using transformer-based models. These systems improve accuracy and usability in real-world applications.

D. Conclusion from Literature

Existing research confirms that AI summarization improves efficiency, reduces effort, and enhances academic performance.

IV. GAP ANALYSIS

Despite advancements, existing systems have several limitations:

- Lack of student-focused summarization
- No integrated platform for sharing and storing notes
- Absence of structured academic outputs
- Limited database support for history and retrieval

The proposed system addresses these gaps by providing summarization, storage, and sharing in one platform.

V. PROBLEM STATEMENT

Students face difficulty in finding reliable and organized study notes. Existing resources are scattered and unstructured, leading to inefficient learning.

Manual summarization is time-consuming, and existing tools do not integrate note sharing with summarization. Therefore, there is a need for a centralized system that combines both functionalities.

VI. OBJECTIVES

- Develop an AI-based summarization system
- Extract key points and important concepts
- Provide a user-friendly interface
- Store summaries for future access
- Improve learning efficiency and revision speed

VII. TOOLS AND TECHNOLOGIES

A. Backend

We have used Node.js to develop the backend of the Smart Notes Summarizer system. Node.js is an open-source JavaScript runtime environment that allows server-side programming using JavaScript. It is fast and efficient because it follows a non-blocking and event-driven architecture, which helps in handling multiple user requests smoothly.

Reasons for Selecting Node.js and Express 1. Fast and efficient performance 2. Easy API development and routing 3. Scalable backend structure 4. Suitable for real-world web applications

B. Frontend

For the frontend of our project, we have used EJS (Embedded JavaScript Templates). EJS helps in creating dynamic and interactive web pages by allowing JavaScript logic inside HTML templates. It makes the project simple because it works directly with Express and does not require complex frontend frameworks.

Reasons for Selecting EJS 1. Easy to learn and implement 2. Works smoothly with Express.js 3. Supports dynamic page rendering 4. Lightweight and beginner-friendly

C. Database

We have used MongoDB as the database system for storing uploaded notes, generated summaries, key points, and related user information. MongoDB is an open-source, powerful, and scalable NoSQL database that stores data in the form of JSON-like documents (BSON format). It is highly suitable for our Smart Notes Summarizer project because academic notes and AI-generated summaries can vary in length and structure. MongoDB allows flexible storage, fast retrieval, and efficient management of user history and summarized content in an organized way. MongoDB ensures high performance, easy integration with Node.js/Express, and supports quick access to saved summaries for future revision.

Reasons for Selecting MongoDB 1. Flexible NoSQL database (stores data in document format) 2. Handles unstructured and long text data easily (notes and summaries) 3. Fast searching and retrieval using indexing and queries 4. Scalable and efficient for storing multiple users' notes and summary history 5. Easy integration with web applications using Node.js and Mongoose

D. AI Integration

To generate summaries automatically, we have used a Generative AI API specifically for text summarization. This AI technique understands large academic content and produces a short, meaningful summary along with important key points. It reduces the need for manual reading and helps students revise faster.

Reasons for Using Generative AI Summarization 1. Produces meaningful and accurate summaries 2. Saves students' time and effort 3. Extracts key points for quick revision 4. Works well with large academic documents

VIII. METHODOLOGY

The system follows a structured workflow:

A. Document Upload

The first step of the system is to take input from the user. The user can upload academic documents such as PDF files or text-based notes through a simple web interface created using EJS. Once the document is uploaded, the backend validates the file type and size to ensure that only supported academic documents are processed. This step is important because it ensures smooth user interaction and prevents unsupported or corrupted files from breaking the system.

B. Text Extraction

After the document is uploaded successfully, the backend extracts the textual content from the file. For typed and digital PDFs, the system reads the document and converts it into plain text format. This extracted text becomes the main input for the summarization module. If the document contains too much unnecessary formatting or extra spaces, the system performs basic preprocessing such as:

- Removing extra line breaks
- Cleaning unwanted symbols
- Converting content into a readable text format

This step ensures the AI receives clean and meaningful input for better summarization results.

C. AI Summarization

Once the text is extracted, the system sends the content to a Generative AI API for summarization. This is the core functionality of the project. The AI analyzes the academic text and produces:

- A short summary (easy to understand)
- Key points / bullet points for quick revision

The AI is guided using a structured prompt so that the output remains academic-focused, clear, and not too lengthy. This makes the summarizer suitable for exam preparation and quick learning.

D. Result Presentation

After receiving the AI-generated summary, the backend formats the result properly and sends it to the frontend. The summary and key points are displayed on the result page using EJS templates. This makes the output:

- Clean and readable
- Properly structured
- Easy to revise in less time

The system also provides options like viewing past summaries and re-checking important notes.

E. Data Storage

To make the system more useful, the generated summary is stored in PostgreSQL. The database stores information such as:

- Document title / file name
- Extracted summary
- Key points
- Date and time of summarization

Storing this data helps the user access summary history anytime without uploading the same document again. It also makes the project more complete and practical for real-world academic use.

F. Retrieval

The stored summaries can be retrieved from the PostgreSQL database whenever required. Users can view previously summarized notes, which helps in:

- Fast revision before exams
- Maintaining an organized study record
- Saving time and effort

This feature improves the usability of the system and adds value compared to one-time summarization tools.

IX. SYSTEM ARCHITECTURE

- 1) Start
- 2) Student Registration / Login
- 3) Upload Typed Notes (PDF / Text)
- 4) System Validates File
 - If Invalid File → Reject Handwritten / Scanned Notes
 - If Valid File → Proceed to next step
- 5) Store Notes in Database
- 6) AI Module Extracts Text
- 7) AI Generates Summary & Tags
- 8) Notes Categorized by Subject / Topic
- 9) Notes Shared on Platform
- 10) Other Students Search Notes
- 11) View Notes
- 12) Rate & Give Feedback
- 13) Save Favorite Notes
- 14) End

X. PLANNING AND DESIGN

Key features such as user authentication, note uploading, AI summarization, quiz generation, and performance tracking were outlined. A feasibility analysis was carried out to evaluate the available resources and tools, leading to the selection of the DeepSeek API for AI-related tasks and a full-stack development framework comprising React, Node.js, Express, and Mongo DB. This phase also included breaking down the project into manageable tasks that could be implemented over several Agile sprints.

The Login Page is the entry point of the application and plays a crucial role in ensuring secure access to the system. It authenticates users based on their registered email ID and grants access only to authorized individuals. The login module is integrated within the overall system architecture, where the React frontend communicates with the Node.js backend for verification. UI wireframes were created to visualize the layout of major components, including the dashboard, upload section, summary viewer, and quiz page. During this stage, the backend was also designed, encompassing the design of the database schema and the design of key components such as file upload, generated summaries, and quiz management. API endpoints were outlined to ensure smooth interaction between the frontend, backend, and AI services. The design stage ensured that every module was logically structured and ready for incremental development. Fig. 2(a) displays the upload notes interface. User needs to press the “Choose file” button to upload a file from their devices. The system then generates a summary of the notes and displays it on the summary preview page as illustrated in Fig. 2(b). The left panel displays the original uploaded content, while the right panel presents the generated summary content. Additionally, “Save” and “Copy” buttons are provided, which allow users to save the summary in their personal database or copy the summarized text for external use.

XI. EVALUATION

Testing was conducted using:

- **Functional Testing:** The system was tested to ensure that all features such as user registration, login, file upload, text extraction, and AI-based summarization work correctly. Each module was verified against the expected output.
- **Performance Testing:** The performance of the system was evaluated based on response time, especially during file upload and summary generation. The system was tested with different file sizes to ensure smooth and efficient processing.
- **User Acceptance Testing:** The system was tested by real users (students) to evaluate usability and effectiveness. Feedback was collected to ensure the system meets user requirements and provides a satisfactory user experience.

Testing was conducted throughout the development to verify the accuracy and reliability of system functions. Black box testing was applied to both frontend and backend components. AI-generated summaries and quizzes were reviewed

for relevance and clarity. Performance tracking features were also tested to evaluate system responsiveness and processing efficiency. This continuous testing helped reduce errors and enhance the user experience. Finally, a user acceptance test (UAT) was conducted to ensure that the system not only functions correctly but also meet the educational needs of its target audience. The testing process was divided into two main phases:

1. Developer-led functional and performance testing: Conducted by the developer to validate technical correctness.
2. End-user acceptance testing: Conducted by participants to assess usability and learning value. Results show improved efficiency, accuracy, and usability.

XII. CONCLUSION

The proposed system provides a complete solution for academic note management and summarization. It reduces manual effort, improves productivity, and enhances learning efficiency. Future work includes OCR, multilingual support, and AI-based question generation.

REFERENCES

- [1] All Social Science Journal, "AI-Based Text Summarization for Academic Learning," 2025. [Online]. Available: <https://www.allsocialsciencejournal.com>
- [2] IEEE, "Automatic Text Summarization Techniques," IEEE Xplore, 2018. [Online]. Available: <https://ieeexplore.ieee.org/document/8424747>
- [3] SSRN, "Generative AI for Document Summarization," 2024. [Online]. Available: <https://ssrn.com>
- [4] A. Vaswani et al., "Attention Is All You Need," NIPS, 2017.
- [5] J. Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers," NAACL, 2019.